High-performance logging with source generators

Source generators are a compiler feature introduced in C# 9. Using this feature, you can automatically generate boilerplate code when your project compiles. .NET 7 includes several built-in source generators, such as the `Regex` generator I described in chapter 14. There's also a source generator that works with `ILogger`, which can help you avoid pitfalls such as accidentally using interpolated strings, and makes more advanced and performant logging patterns easy to use.

To use the logging source generator in the `OnGet` handler from listing 26.2, define a *partial method* in the `IndexModel` class, decorate it with a `[LoggerMessage]` attribute, and invoke the method inside the `OnGet` handler method:

```
[LoggerMessage(10, LogLevel.Information, "Loaded {RecipeCount} recipes")]
partial void LogLoadedRecipes(int recipeCount);

public void OnGet()
{
    Recipes = _service.GetRecipes();
    LogLoadedRecipes(Recipes.Count);
}
```

The `[LoggerMessage]` attribute defines the event ID, log level, and message the log message uses, and the parameters of the partial method it decorates are substituted into the message at runtime. This pattern also comes with several analyzers to make sure you use it correctly in your code while optimizing the generated code behind the scenes to prevent allocations where possible.

The logging source generator is optional, so it's up to you whether to use it. You can read more about the source generator, the extra configuration options, and how it works on my blog at http://mng.bz/vn14 and in the documentation at http://mng.bz/4D1j.

Not every log message will have all the possible elements. You won't always have an `Exception` or parameters, for example, and it's common to omit the `EventId`. There are various overloads to the logging methods that